

Tests d'Intégration et Validation d'Infrastructure

Contrairement aux tests unitaires vus précédemment, les **tests d'intégration** interagissent avec la véritable infrastructure. Il n'y a **aucun Mock** ici. Votre code va interroger le véritable système d'exploitation, le véritable disque dur et le véritable réseau.

Dans un cycle DevOps, ces tests sont exécutés par la pipeline juste après le déploiement (Phase de Validation de Staging ou de Production) pour garantir que le serveur est conforme et opérationnel.

Vous allez utiliser Pester 5 pour concevoir une sonde de validation système et réseau (Audit de Conformité).

Exercice 1 : Préparation de l'Environnement et Tagging

Dans un projet professionnel, les tests unitaires et les tests d'intégration vivent dans le même dépôt, mais ne s'exécutent pas au même moment. Il faut pouvoir les différencier.

1. Créez un nouveau fichier nommé Validation-Serveur.Tests.ps1 dans votre dossier C:\Audit\.
2. Déclarez le bloc racine Describe.
3. Ajoutez un **Tag** à ce bloc pour préciser qu'il s'agit d'un test d'intégration.

Aide syntaxique :

```
Describe "Validation de la configuration du serveur (Post-Déploiement)" -Tag "Integration",  
"Infrastructure" {  
    # Vos tests iront ici  
}
```

Exercice 2 : Validation de la Couche Système (Services)

Vous allez auditer l'état réel de services critiques sur votre poste/serveur local.

1. Définissez un bloc Context "Couche Système et Services".
2. **Test du Spouleur d'impression (ou WinRM) :**
Rédigez un bloc It nommé "Le service Spooler doit être en cours d'exécution".
À l'intérieur, récupérez la propriété Status du service Spooler avec la commande native Get-Service et affirmez qu'elle doit être égale à "Running".
3. **Test du mode de démarrage :**

Rédigez un deuxième bloc It nommé "Le service WinRM doit être configuré en démarrage Automatique".

À l'intérieur, récupérez le mode de démarrage de WinRM (propriété StartType) et affirmez qu'il vaut "Automatic".

Exercice 3 : Validation de la Couche Réseau (Ports et Connectivité)

Un service qui tourne n'est utile que s'il écoute correctement sur le réseau.

1. Définissez un bloc Context "Couche Réseau et Connectivité".
2. **Test d'écoute locale :**
Rédigez un test "Le port 5985 (WinRM HTTP) doit être ouvert et en écoute locale".
Utilisez la commande `Test-NetConnection -ComputerName localhost -Port 5985`.
Vérifiez que la propriété `TcpTestSucceeded` retournée vaut bien `$true`.
3. **Test de résolution DNS externe :**
Rédigez un test "Le serveur doit pouvoir résoudre les adresses DNS externes (ex: google.com)".
Utilisez `Resolve-DnsName -Name google.com -ErrorAction SilentlyContinue` et vérifiez que le résultat n'est pas nul ou vide (`Should -Not -BeNullOrEmpty`).

Exercice 4 : Validation de la Sécurité et du Système de Fichiers

1. Définissez un bloc Context "Couche Stockage et Sécurité".
2. **Test de présence de répertoires :**
Rédigez un test "Le dossier de déploiement `C:\CorporateShares` doit exister".
(*Si le dossier n'existe pas sur votre poste, créez-le manuellement avant de lancer le test pour simuler un serveur pré-configuré*).
Utilisez `Test-Path` et vérifiez la réponse.
3. **Test de politique d'exécution :**
Rédigez un test "La politique d'exécution PowerShell doit autoriser les scripts distants (`RemoteSigned` ou `AllSigned`)".
Utilisez `Get-ExecutionPolicy` et vérifiez que la valeur retournée figure parmi une liste autorisée avec l'opérateur `-in`.
Aide syntaxique :
`$Politique = Get-ExecutionPolicy`
`$Politique | Should -BeIn @("RemoteSigned", "AllSigned", "Bypass")`

Exercice 5 : Exécution Sélective des Tests

Il est temps de lancer vos tests en simulant le comportement de l'orchestrateur CI/CD.

1. Exécutez **uniquement** les tests marqués par le tag "Integration". Ouvrez votre console et tapez :
Invoke-Pester -Path "C:\Audit\Validation-Serveur.Tests.ps1" -Tag "Integration" -Output Detailed
2. Modifiez volontairement la configuration de votre machine pour faire échouer un test (ex: Arrêtez le service Spooler).
3. Relancez la commande Invoke-Pester et constatez comment Pester identifie la défaillance de l'infrastructure de manière très lisible.